

API Platform & Infrastructure

1. Feature Name

API Platform & Infrastructure

2. Purpose

- **Problem Solved:** The platform needs a consistent, secure, observable HTTP API surface: unified response envelopes, centralized exception handling, request logging, rate limiting, environment-based secrets, Swagger docs, and standardized model validation.
 - **Why It Exists:** A cross-cutting infrastructure layer keeps every feature consistent (same `ApiResponse` envelope, same error semantics) and safe (secrets out of source control, retries, rate limits, role policies).
 - **Role in Application:** It is the foundation everything else runs on — hosting, configuration, middleware pipeline, DI registration, and API documentation.
-

3. What the User Can Do

- Call a REST API under `/api/v1/*` that always returns the same JSON envelope (`{ success, message, data, errors, timestamp }`).
 - Get **human-readable validation errors** instead of raw `ValidationProblemDetails` when model binding fails.
 - Rely on predictable error semantics: 400 (bad input), 403 (forbidden), 404 (not found), 409 (dependency conflict), 429 (rate limited), 500 (unexpected).
 - Browse interactive Swagger UI at `/swagger` (development) with a **Bearer** authorization scheme.
 - Have their actions rate-limited per IP (50 req/min, health endpoint exempt) with `X-RateLimit-*` headers.
-

4. Feature Workflow

[Request arrives]

- `ExceptionHandler` (outermost) – converts **any** unhandled exception **to** the `ApiResponse` envelope
 - `LoggingMiddleware` – logs **method/path/status/duration**
 - `RateLimitingMiddleware` – IP-based sliding **window** (50/min), **429** when exceeded
 - (dev) Swagger UI route handling
 - HTTPS redirect (non-dev) + CORS policy
 - Authentication (JWT bearer) → **Authorization** (roles/policies)
 - Controller endpoint → service/repository → JSON response via `ApiResponse`
 - ModelState failures → `InvalidModelStateResponseFactory` → `ApiResponse.Fail` (400)
-

5. How It Works Internally

Unified Response Envelope (`Common/ApiResponse.cs`)

`ApiResponse<T>` has `Success`, `Message`, `Data`, `Errors`, `Timestamp`. `Ok(data, msg)` sets `Success=true`; `Fail(msg, errors)` sets `Success=false`. Every controller returns `Ok(ApiResponse<T>.Ok(result))` or a `Fail` variant, so the client always parses one shape.

Exception Middleware (`Middleware/ExceptionMiddleware.cs`)

Catches everything and maps by exception type:

- `UnauthorizedAccessException` → **403** with `ex.Message`.
- `KeyNotFoundException` → **404**.
- `ArgumentException` → **400**.
- `DbUpdateException` with a SQL FK violation (`SqlException` number 547, walked via `InnerException` chain) → **409** with a safe dependency message — never leaks constraint text.
- Anything else → **500** with a generic message (full detail logged).

If the response has already started, it rethrows rather than writing a broken body.

Logging Middleware (`Middleware/LoggingMiddleware.cs`)

A `Stopwatch` wraps the pipeline; logs `"{Method} {Path} responded {StatusCode} in {Elapsed}ms"` as `Information`.

Rate Limiting (`Middleware/RateLimitingMiddleware.cs`)

- In-memory sliding window keyed by `RemoteIpAddress` (`ConcurrentDictionary`), max **50 requests / minute**.
- `/api/v1/health` bypasses the limiter.
- Emits `X-RateLimit-Limit` and `X-RateLimit-Remaining` headers.
- On exceed → 429 with `{ success:false, message:"Too many requests..." }`.
- Note: per-process (not distributed) — acceptable for a single-instance deployment.

Secure Configuration & `.env` (`Program.cs`)

- Loads `.env` (via `DotNetEnv`) if present, mapping `LEARNPATH_*` vars → config keys (AI key/model/URL, JWT secret, DB connection).
- Fails fast (`InvalidOperationException`) listing any missing required vars.
- Overrides `appsettings.json` via `AddInMemoryCollection`; secrets are never committed.

Model Validation Normalization

`ConfigureApiBehaviorOptions` overrides `InvalidModelStateResponseFactory`: `ModelState` errors are flattened into a `List<string>` (using `ErrorMessage`, falling back to the exception message), joined into one message, and returned as `ApiResponse<object>.Fail(message, errors)` → 400. `FluentValidation` validators (`AddValidatorsFromAssemblyContaining<Program>`) run per-DTO.

DI & Architecture

Scoped services for each domain (`IAuthService`, `IDashboardService`, `ICommunityService`, `IAuditLogService`, `IAdminService`, ...), the `CommunityRepository`, **AI stack** (`NvidiaProvider` via `AddHttpClient`, `AiFeedbackService`, `PromptBuilder`, `AiResponseParser`), `IHttpContextAccessor`, **AutoMapper**, and `UploadSettings`.

Authorization Policies

`AddAuthorizationBuilder: AdminOnly` (role `Admin`) and `InstructorOrAdmin` (`Admin` or `Instructor`) – used alongside `[Authorize(Roles=...)]` and attribute-level guards.

Swagger

Swagger/OpenAPI v1 with XML doc comments, title "LearnPath API", and a Bearer (JWT) security definition wired into a global security requirement. Enabled only in development.

Seeding & Migrations

On startup: `db.Database.MigrateAsync()`, then `RoleSeeder` and `AdminSeeder` ensure the seed roles and Super Admin account exist.

6. Frontend Implementation

Layer	File Path	Responsibility
Client	<code>frontend/src/services/apiClient.ts</code>	Axios instance with base URL, interceptors for auth headers, response unwrapping, and centralized error handling
Service modules	<code>frontend/src/services/*.ts</code>	Per-feature API clients consuming the envelope

The frontend consumes the unified envelope (reads `data.data`, handles `message / errors`) and centralizes error toasts in `apiClient` interceptors.

7. Backend Implementation

Layer	File Path	Responsibility
Host	<code>backend/Program.cs</code>	.env load, config mapping, Identity/JWT/CORS/DI/Swagger/middleware pipeline, seeding
Middleware	<code>backend/Middleware/ExceptionMiddleware.cs</code>	Exception → status/envelope mapping
Middleware	<code>backend/Middleware/LoggingMiddleware.cs</code>	Request/response timing logs

Layer	File Path	Responsibility
Middleware	backend/Middleware/RateLimitingMiddleware.cs	IP rate limiting (50/min)
Common	backend/Common/ApiResponse.cs	Unified response envelope

8. Database Implementation

No feature-specific tables. Infrastructure touches:

- `AspNetUsers`, `AspNetRoles` (Identity) — seeded by `RoleSeeder` / `AdminSeeder` after `MigrateAsync()`.
- `ApplicationDbContext` migrations run automatically at startup.

9. Security & Authorization

- JWT bearer auth with full token validation (issuer, audience, lifetime, signing key, zero clock skew).
- Identity password policy (digit, lower, upper, min 8; 5-attempt lockout for 5 minutes; unique email).
- CORS restricted to `AllowedOrigins` from configuration; credentials allowed.
- Rate limiting to blunt abuse (50 req/min/IP, health exempt).
- Secrets managed via `.env` + `AddInMemoryCollection` overrides; required-vars fail-fast check.
- Error responses never leak SQL constraints or stack traces; FK violations return a safe 409 message.
- HTTPS redirect enforced outside development.

10. Important Business Rules

1. **Every response** uses the `ApiResponse<T>` envelope.
2. **Exception → HTTP mapping:** `ArgumentException`=400, `UnauthorizedAccessException`=403, `KeyNotFoundException`=404, FK violation=409, default=500.
3. **Rate limit:** 50 requests/min/IP; `/api/v1/health` exempt; headers `X-RateLimit-Limit/Remaining`.
4. **ModelState failures** become 400 with flattened human-readable errors.
5. **Required .env vars** (`LEARNPATH_NVIDIA_API_KEY`, `LEARNPATH_JWT_SECRET`, `LEARNPATH_DB_CONNECTION`) cause startup failure if missing.
6. Swagger is development-only; HTTPS redirect is production-only.
7. Migrations + seeders run automatically at startup.
8. Rate limiter is in-memory (single-instance scope).

11. Example of Internal Execution

Step 1: Valid request

GET /api/v1/dashboard with a bearer token → LoggingMiddleware records timing → RateLimitingMiddleware increments the caller's window → auth validates the JWT → controller returns ApiResponse.Ok(stats).

Step 2: Invalid model state

POST /api/v1/auth/register with a bad email → ASP.NET ModelState fails → InvalidModelStateResponseFactory returns 400 ApiResponse.Fail("'Email' is not a valid email address.", [errors]).

Step 3: Dependency conflict

DELETE /api/v1/admin/classrooms/{id} on a classroom with assignments → EF throws DbUpdateException → ExceptionMiddleware detects SqlException 547 → 409 "The record could not be deleted because other data depends on it..."

Step 4: Rate limited

A client fires 51 requests in a minute → the 51st gets 429 with X-RateLimit-Remaining: 0.

Step 5: Startup validation

Missing .env → InvalidOperationException lists the missing variables and startup aborts.

12. Files Involved

Backend

- backend/Program.cs
- backend/Middleware/ExceptionMiddleware.cs
- backend/Middleware/LoggingMiddleware.cs
- backend/Middleware/RateLimitingMiddleware.cs
- backend/Common/ApiResponse.cs
- backend/Configuration/* (JwtSettings, UploadSettings, AiOptions)
- backend/Data/Seeders/RoleSeeder.cs, AdminSeeder.cs

Frontend

- frontend/src/services/apiClient.ts
- frontend/src/services/*.ts (per-feature clients)

13. Functionality

- Unified ApiResponse envelope for all endpoints
- Centralized exception-to-status mapping (400/403/404/409/500)
- Safe FK-violation handling without leaking SQL details
- Request logging with timing
- IP-based rate limiting with response headers (health endpoint exempt)

- `.env`-driven secure configuration with fail-fast validation
- Normalized model-validation errors (400 + readable list)
- FluentValidation auto-registration
- JWT bearer auth + role authorization policies
- Restricted CORS and HTTPS redirect in production
- Swagger UI with Bearer auth (development)
- Automatic migrations + role/admin seeding on startup